

DAY 3



</>

31 Days of React Design Patterns

Container-Presenter Pattern

Task Implementation



Building a Product List with:



Data
Fetching



Cart
Management



Error
Handling



Reusable
Presenters



The Challenge

Building a real-world product feature without turning everything into one giant component.

Build a product listing feature where:



Products come from an API



Users add items to cart



Errors are handled gracefully

Without creating one giant component.

ProductPage.jsx

- Data fetching
- Cart state
- Rendering
- Events
- Error handling
- UI updates
- Everything else...



One component doing everything becomes difficult to maintain.



Bloated Components



Mixed Responsibilities



Harder Testing



Poor Reusability



Requirements

Define responsibilities before writing components.

Container Responsibilities



- ✓ API Calls
- ✓ Loading State
- ✓ Error Handling
- ✓ Cart Logic



```
const fetchProducts = () => {  
  // API call  
  // handle loading  
  // handle error  
  // manage cart  
}
```



Logic



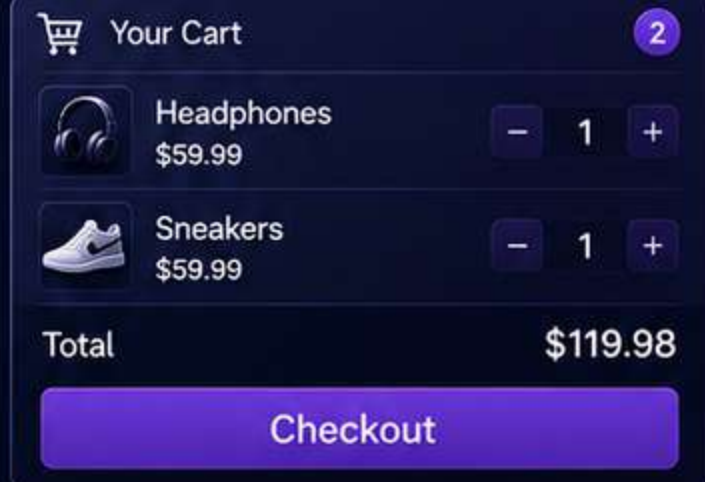
UI



Presenter Responsibilities



- ✓ Product Rendering
- ✓ User Interactions



Smaller Presenters

- ProductCard
- CartItemCard



Smaller components create **cleaner architecture**.



Clear
Roles



Better
Reuse



Easy
Testing

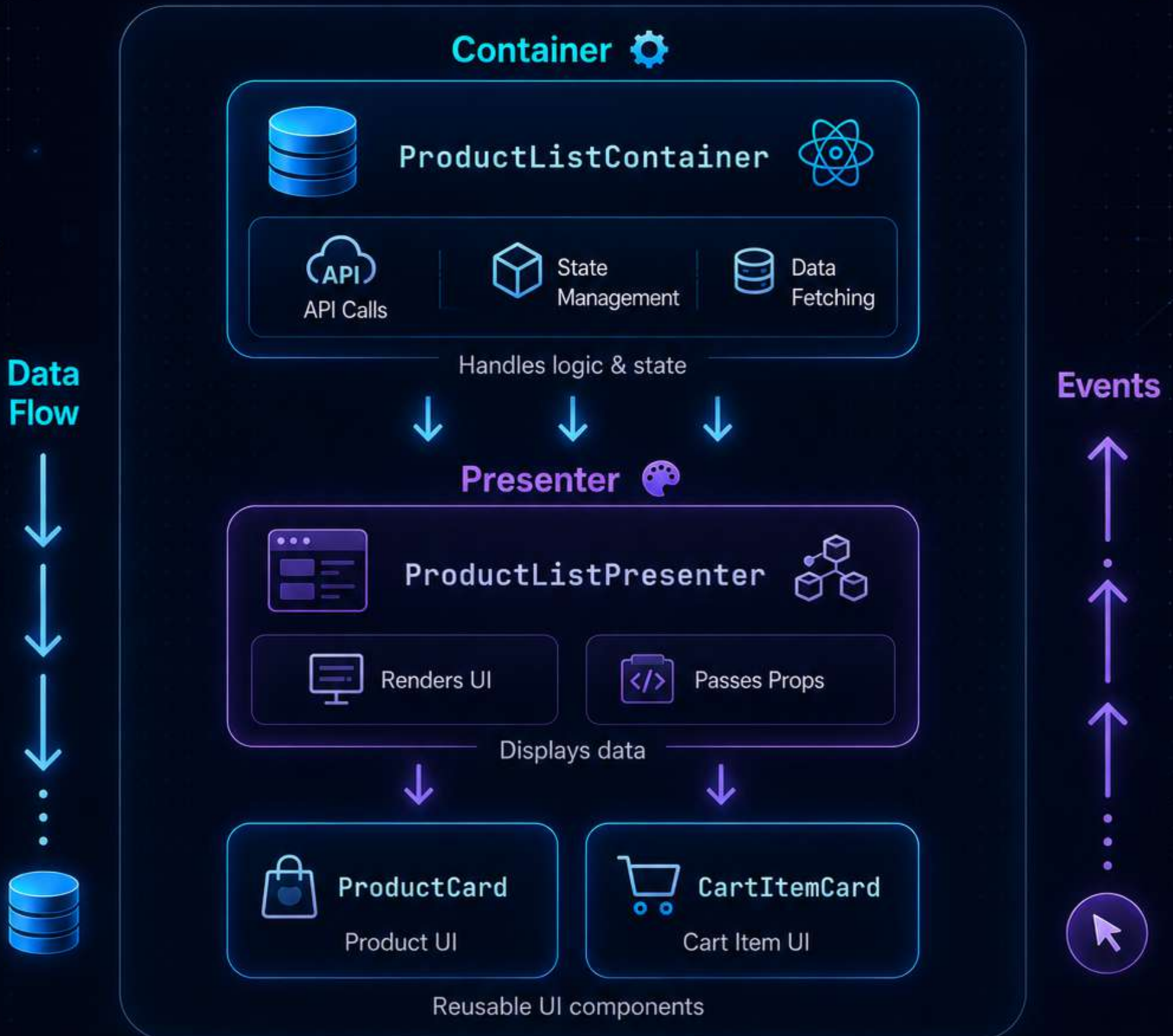


Maintainable
Code



Architecture Overview

Separate business logic from presentation through a clear component hierarchy.



Clear architecture creates **maintainable applications.**



Separation of Concerns



Reusable Components



Predictable Data Flow



Easy Maintenance



Implementation Approach

{ }

Divide responsibilities so each component has a single purpose.

</>



Container

- ✓ Fetch Products
- ✓ Store Cart State
- ✓ Handle Add-to-Cart
- ✓ Pass Props Down

 State

 Logic

 Decisions

Brain



Face



Presenter

- ✓ Render Products
- ✓ Render Cart
- ✓ Trigger Callbacks

 UI

 Layout

 Display

 Container		 Presenter
 Thinks	→	 Displays
 State	→	 UI
 Logic	→	 Layout

Think about data. **Display** the experience.



Single Responsibility



Clean Components



Better Reuse



Easy Testing

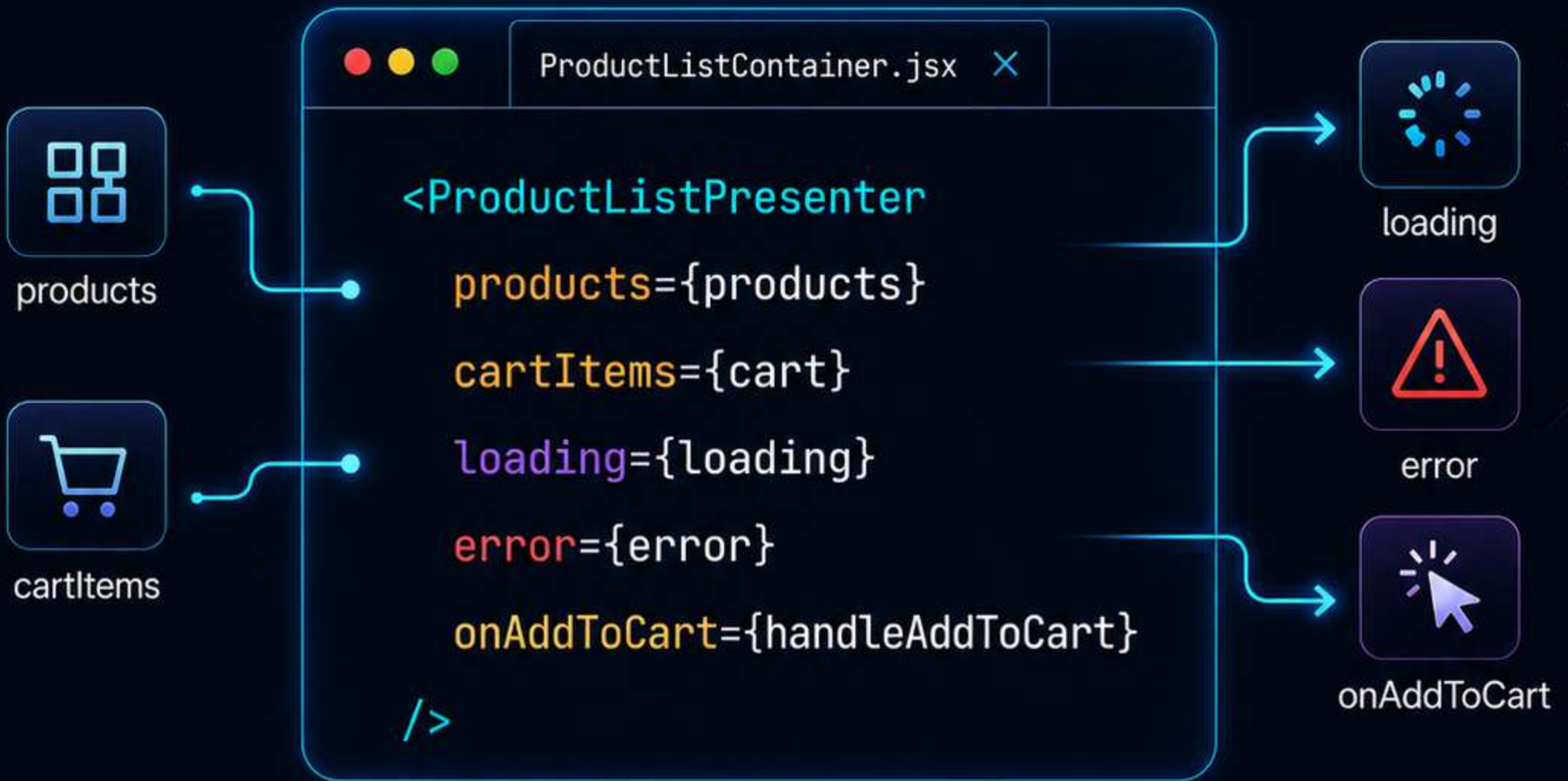


Important Code Snippet

{ }

The container knows everything. The presenter knows nothing.

</>



The Handoff



The **Container** knows everything.
The **Presenter** knows nothing.





Challenges Faced

The hardest part wasn't writing code it was deciding who should own the responsibility.



Confusion



Clarity



Key Realization

I initially wanted the presenter to update cart state, handle API errors, and fetch data. That quickly **blurred responsibilities**.



The Pattern Forced Better Decisions



Separation

Responsibilities stay clear.



Simplicity

Components stay focused.



Maintainability

Code becomes easier to evolve.



Lessons Learned



Patterns become valuable when projects become larger and more complex.



1 Easier Testing



Smaller responsibilities make components easier to verify.

2 Smaller Components



Focused components reduce complexity.

Clear Separation



Long-Term Benefits

3 Better Reuse



Reusable pieces save time and effort.

4 Less Mental Overhead



Clear boundaries make code easier to understand.



Key Insight

Patterns become valuable when projects grow.

Good architecture isn't about writing more code. It's about making code easier to understand, **maintain, and extend.**



Maintainability



Scalability



Readability



Reusability



Project Result



Success criteria achieved through
clear separation of responsibilities.



Success Criteria Achieved

1 No JSX in Container



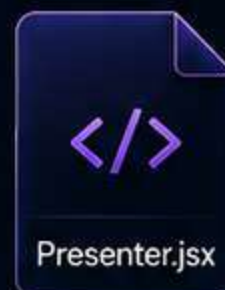
Container.jsx

Container

- Fetch Data
- Manage State
- Handle Logic

Business logic
without UI rendering.

2 No Business Logic in Presenter



Presenter.jsx

Presenter

- Render UI
- Handle Events
- Display Data

Pure presentation
and interactions.



Logic



Props



UI

3 Reusable Cards



ProductCard



CartItemCard



\$ 29.99

Modular and reusable
UI components.

4 Clear Separation of Concerns



Container
(Think)

Data / State

Props

UI



Presenter
(Display)

Predictable and
maintainable structure.



Maintainability



Reusability



Clarity



Separation



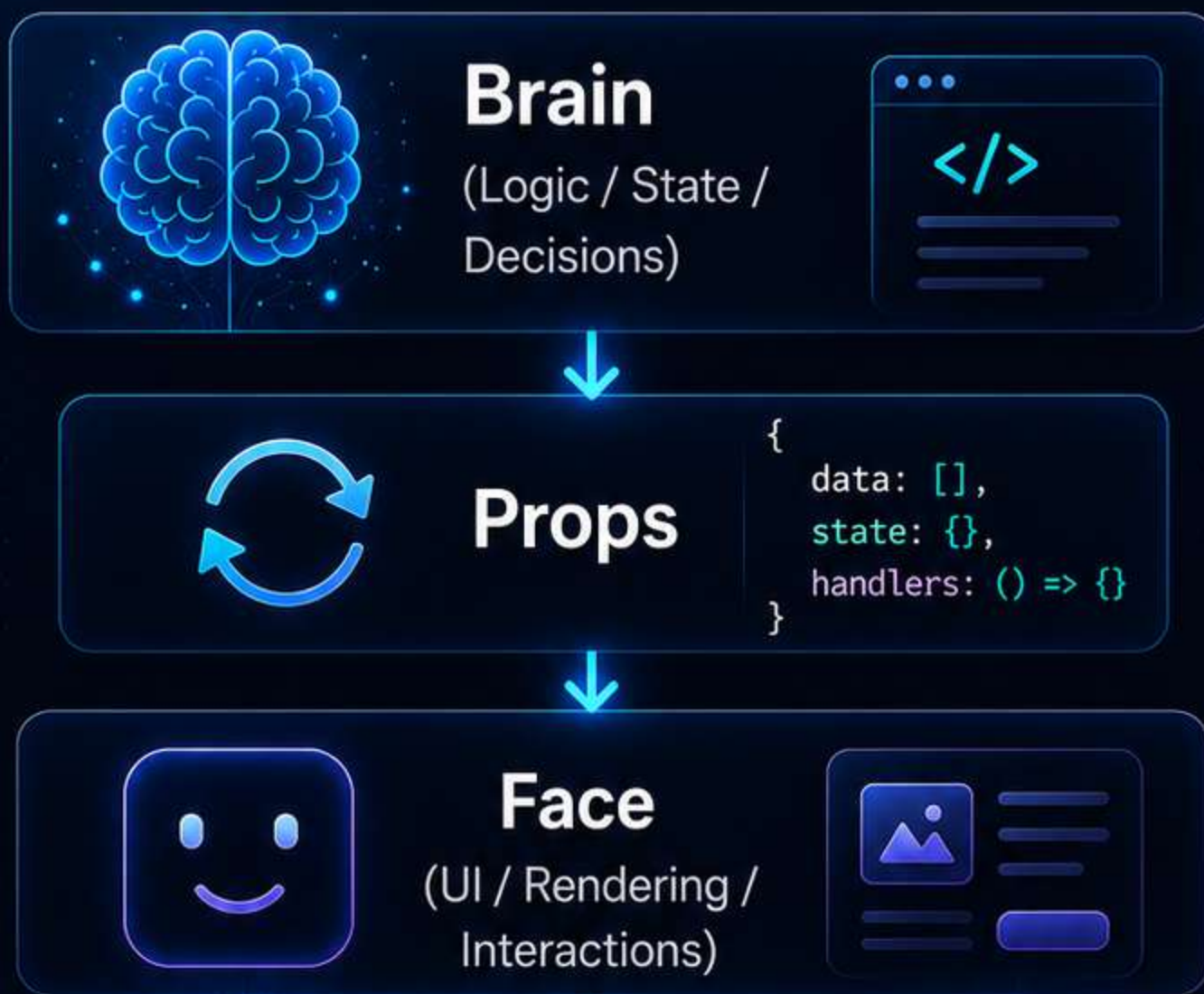
Good architecture isn't about
more components. It's about
clearer responsibilities.





Key Takeaway

Great components don't try to do everything.



“ A component should either **decide what happens** or **decide how it looks** — not both. ”



Maintainable

Easy to update and extend.



Reusable

Use components across features.



Testable

Logic and UI are isolated.



Scalable

Grows with your application.



mtayyabrawan

Let's Build Performance-Optimized Websites

Frontend Developer • React & Next.js Developer